



Lecture 05: The Anatomy of a Svelte Component

INTERVIEW QUESTION

Describe the typical anatomy and structure of a single-file .svelte component.

Now that we understand Svelte is a compiler that produces surgically precise DOM updates, it is time to look at the source material: the `.svelte` file itself.

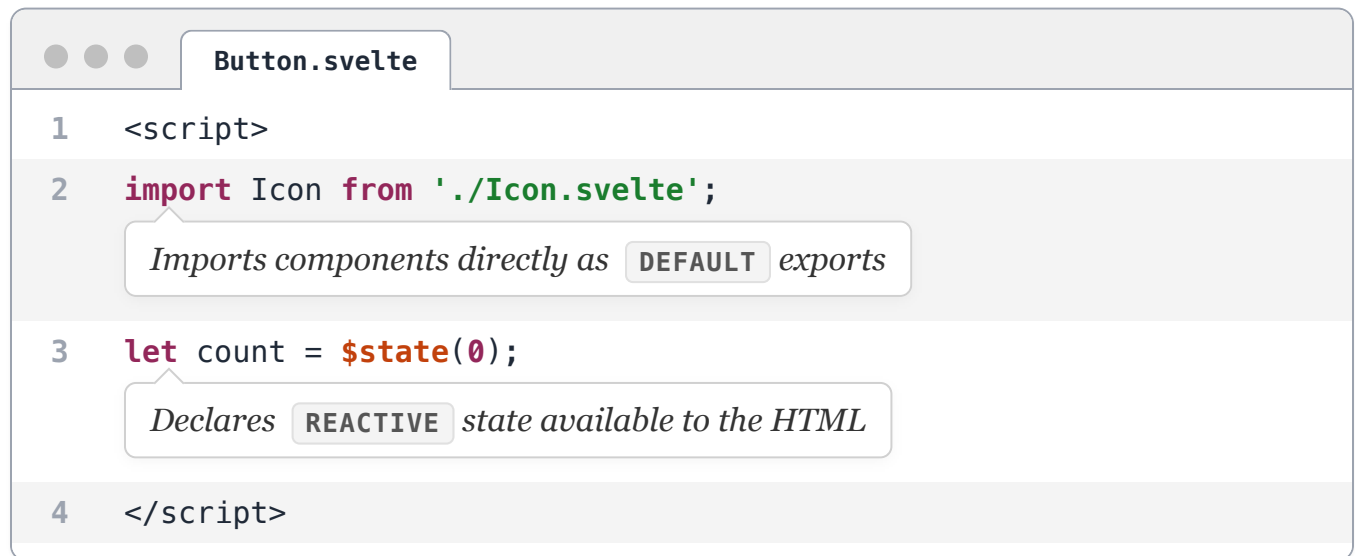
If you have used Vue, the concept of a Single-File Component (SFC) will be very familiar. Svelte embraces the SFC pattern, meaning a single `.svelte` file encapsulates everything a component needs to function: its logic, its styling, and its markup.

The structure of a `.svelte` file is incredibly simple and closely mirrors standard HTML. There is no boilerplate, no `export default class`, and no required `render()` method wrapper. It is broken into three distinct sections. The community standard usually places `<script>` at the top, followed by the markup, and `<style>` at the bottom.

1. The Logic (`<script>`)

At the top of the file, you place your JavaScript (or TypeScript) inside a standard `<script>` tag.

This block runs exactly once when the component is instantiated. It is where you declare your reactive state (using `$state`), define your functions, and import other components or libraries. Any variable declared at the top level of this script is automatically made available to the markup below it.



The screenshot shows a code editor window titled "Button.svelte". The code is as follows:

```
1 <script>
2 import Icon from './Icon.svelte';
3 let count = $state(0);
4 </script>
```

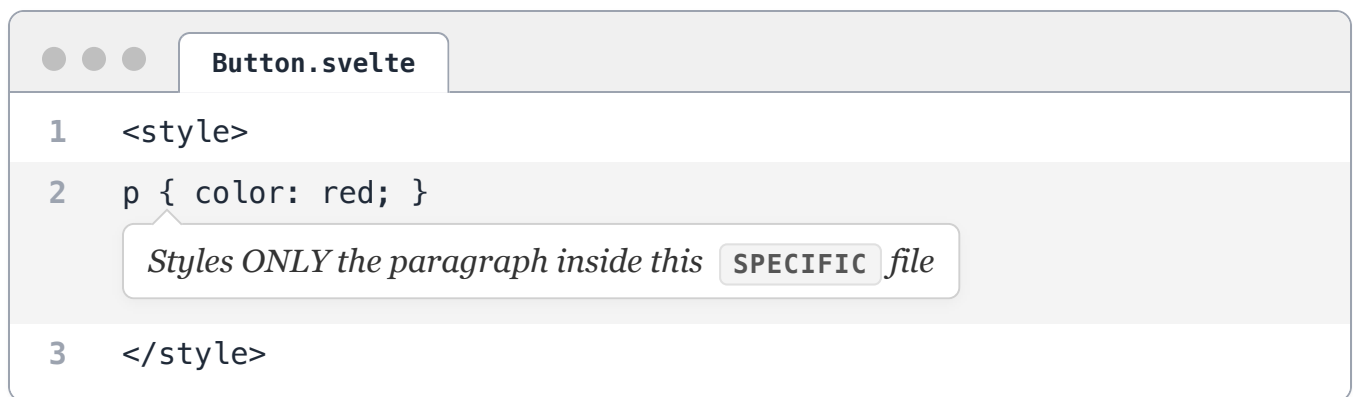
There are two callout boxes explaining parts of the code:

- A callout box pointing to line 2: *Imports components directly as **DEFAULT** exports*
- A callout box pointing to line 3: *Declares **REACTIVE** state available to the HTML*

2. The Styling (`<style>`)

You write your CSS inside a standard `<style>` tag.

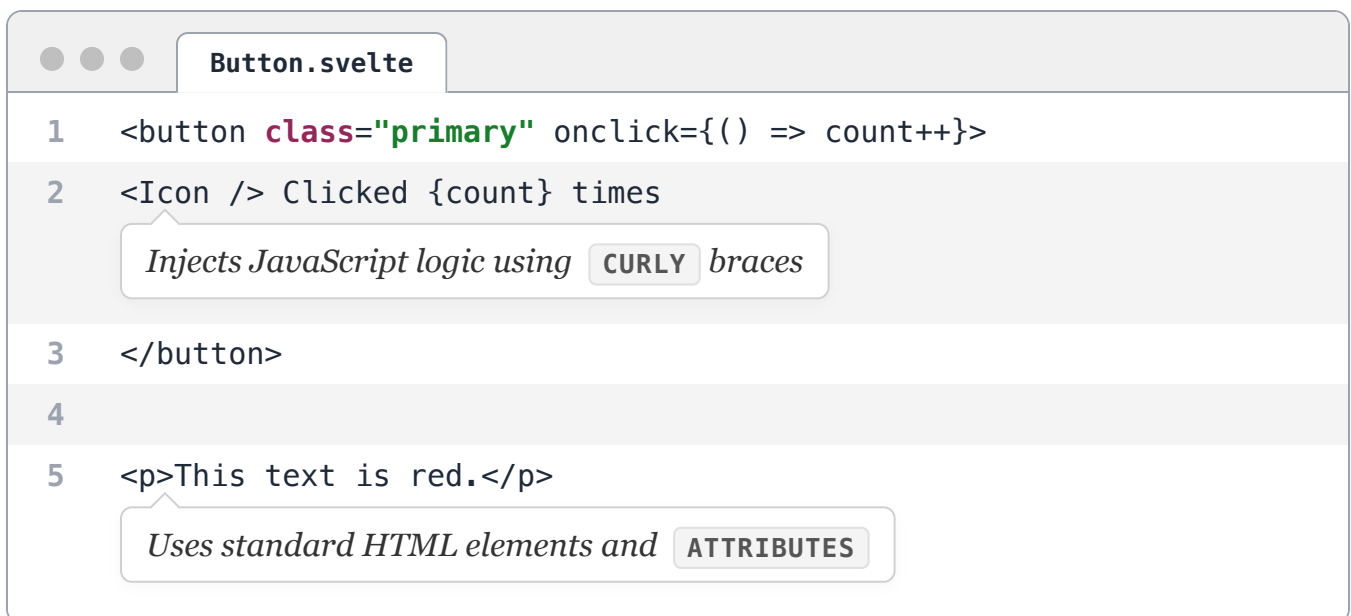
The magic of Svelte's compiler is that all styles are scoped by default. If you write a rule targeting a `p` tag in `Button.svelte`, it will only style `<p>` tags *inside* that specific component. It will not leak out and accidentally ruin the styling of paragraphs elsewhere in your application. The compiler enforces this by hashing a unique class name and applying it to both the CSS and the HTML during the build step.



3. The Markup (HTML)

Everything else in the file is treated as the component's markup. Svelte uses standard HTML augmented with a powerful, specialized templating syntax. You can use single curly braces `{}` to inject JavaScript expressions directly into the DOM or to bind attributes.

Because Svelte embraces standard HTML, you don't need to learn JSX quirks (like using `className` instead of `class` or `htmlFor` instead of `for`). It is just HTML.



```
1 <button class="primary" onclick={() => count++}>
2 <Icon /> Clicked {count} times
3 </button>
4
5 <p>This text is red.</p>
```

*Injects JavaScript logic using **CURLY** braces*

*Uses standard HTML elements and **ATTRIBUTES***

KEY TAKEAWAY

A Svelte component is just an HTML file with superpowers. The `<script>` handles logic, the `<style>` handles scoped presentation, and the rest is standard HTML markup.

Why this architecture wins

By colocating logic, styling, and markup into a single file without wrapping them in complex JavaScript boilerplate, Svelte drastically lowers the cognitive load required to read a component. A new developer can open a `.svelte` file and instantly understand what it does and how it looks, top to bottom.

INTERVIEW TIP

To impress the interviewer: When asked about Svelte's component anatomy, emphasize that Svelte's `<style>` block provides **zero-configuration scoped CSS**. In React, you often have to install and configure third-party libraries (like CSS Modules or Styled Components) to achieve scoping. In Svelte, it is a native feature of the compiler.

Summary

The `.svelte` file is a beautifully simple, standard-compliant approach to building UI components. It requires zero boilerplate, uses native HTML tags, and enforces style isolation automatically, giving developers a highly ergonomic authoring experience.

Section	Purpose	Key Characteristic
<code><script></code>	Component logic and state	Runs ONCE upon component instantiation
<code><style></code>	Component presentation	CSS is automatically SCOPED to the file
Markup	Component structure	Uses standard HTML and <code>{}</code> for LOGIC injection